A template/ approach to Reproducible Generative Research

Research infrastructure that documents itself — a case study for meta-science and science-integrity teams



The problem

- Research groups publish claims about their own tools and pipelines that no one outside the team can re-check.
- Manuscripts describing infrastructure go stale the moment the infrastructure changes underneath them.
- Reproducibility efforts usually stop at the data and code layer — the documentation layer is still hand-maintained and drifts silently.



Why this is a science-integrity problem, not just a tooling problem

- A stale claim about infrastructure is a form of unverified evidence, structurally similar to an uncheckable statistic in a results section.
- Peer review rarely re-derives a paper's own infrastructure description; it is trusted by default.
- Meta-science groups exist precisely to close gaps like this — this deck is itself an instance of the pattern it is describing.



Landscape

Where meta-science tooling stands today

- Electronic lab notebooks and preregistration platforms address the experiment layer, not the infrastructure-description layer.
- Literate-programming tools (R Markdown, Jupyter, Quarto) bind prose to code within one document, but rarely to a whole repository's live state.
- Very few public examples exist of a manuscript that introspects and reports on the software system that produced it, at repository scale.



Adjacent practice this builds on

- Ten Simple Rules for Reproducible Computational Research (Sandve et al., 2013) — the base discipline of tracking exactly what produced each result.
- Good Enough Practices in Scientific Computing (Wilson et al., 2017) — version control, testing, and modular code as baseline hygiene.
- FAIR principles for research software (Lamprecht et al., 2020) — Findable, Accessible, Interoperable, Reusable, extended here to the manuscript layer itself.

What template_template is

- A manuscript that is generated FROM the repository it describes, not written ABOUT it from memory.
- Every metric in the paper — module counts, test counts, pipeline stages — is computed live by template_template's own introspection code.
- One of 25 public exemplar templates in the same monorepo, each independently tested and coverage-gated.

Source: infrastructure/project/public_scope.py

“Autopoietic: the paper regenerates itself from the code it describes.”

— template_template/README.md

Source: projects/templates/template_template/README.md

How it works

Introspection

- src/template_template/core/introspection.py discovers infrastructure/ subpackages, the pipeline.yaml stage DAG, and the public exemplar roster.
- The scan reads the filesystem and YAML directly — it does not depend on any external service or cached snapshot.
- Function-level entry points (discover_infrastructure_modules, discover_projects, load_pipeline_stages_from_yaml) each return typed, testable dataclasses.



Metrics

- metrics.py turns the introspection scan into a dictionary of ${variable} values — the same token convention used across every exemplar's manuscript.
- build_manuscript_metrics_dict computes counts directly; nothing is copy-pasted from a prior run.
- save_metrics_json persists the full dictionary to output/data/metrics.json for downstream inspection.



Injection

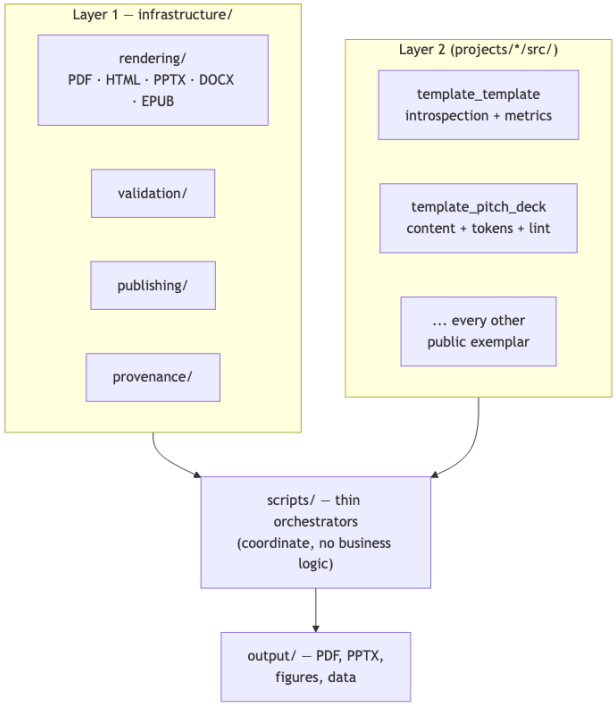
- inject_metrics.py substitutes ${variable} values into manuscript/*.md, writing the resolved text to output/manuscript/ for rendering.
- Unresolved tokens are never silently dropped — a missing variable is a build failure, not a blank space in the PDF.
- Re-running the pipeline regenerates the paper from current repository state — the citation and the artifact cannot drift apart.



Visualization

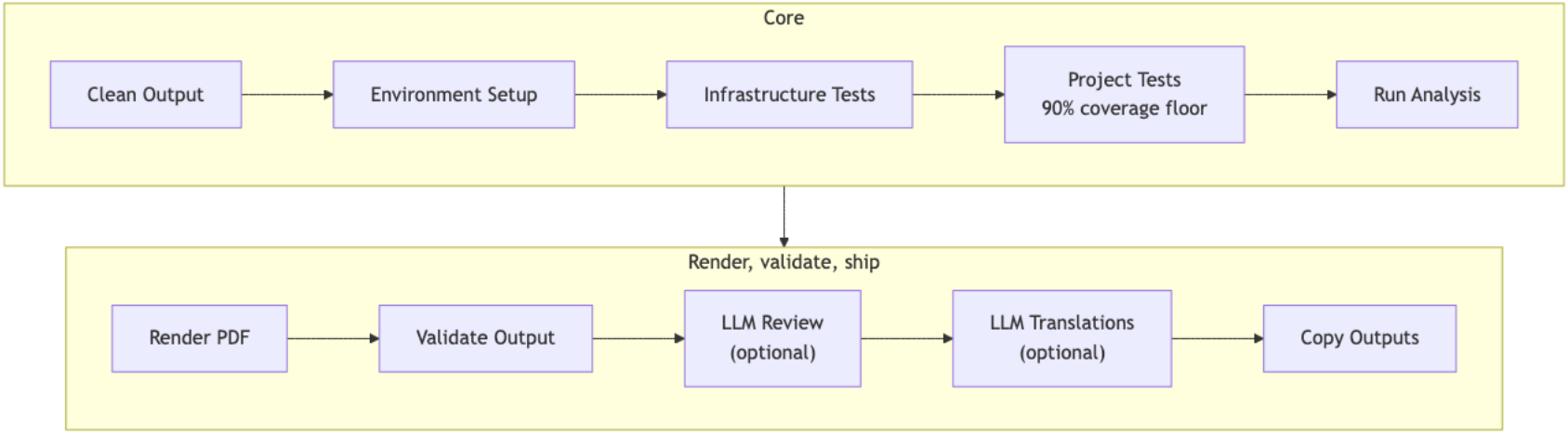
- Four figures — architecture overview, pipeline stages, module inventory, comparative feature matrix — are generated from the same live data, not hand-drawn.
- generate_architecture_viz.py orchestrates dedicated figure modules sharing one palette module, so the visual language stays consistent across all four.
- Regenerating figures after a code change is one script call, not a redraw in an external tool.

Two-layer architecture, at a glance



Source: CLAUDE.md

The core pipeline, stage by stage



Source: CLAUDE.md

Two-layer architecture, in depth

Layer 1 — infrastructure/

- Generic, reusable modules: rendering (PDF/HTML/PPTX/DOCX/EPUB), validation, publishing, provenance, security.
- No project-specific logic lives here — a change to infrastructure/ affects every exemplar identically.
- This deck's own PDF/PPTX renderers (infrastructure/rendering/slide_deck.py, pptx_deck.py) are new additions to this same layer.



Layer 2 — projects/{name}/src/

- Each exemplar's own domain logic: template_template's introspection/metrics, a code-project's optimizer, this deck's own content loader and token/cliche validators.
- Kept out of infrastructure/ by convention, enforced by an automated project/infrastructure-boundary drift check.
- Thin orchestrator scripts (projects/{name}/scripts/) coordinate the two layers but implement no business logic themselves — a size/complexity gate enforces this directly.

Source: projects/templates/template_template/src/template_template

Testing discipline

- No mocks, anywhere: tests use real files, real YAML, real rendered output — never unittest.mock or MagicMock.
- 90% coverage floor on every exemplar's own src/; 60% floor on shared infrastructure/.
- A project fails its own gate the same way regardless of which exemplar it is — there is no per-project exception without an explicit, documented reason.



Proof, not adjectives



Measured, not asserted

153 tests

97.07% coverage on template_template's own source — the same gate every exemplar in the repo must pass. No partial-credit reporting path exists.

Source: docs/_generated/COUNTS.md

Already public

9 historical reference

Prior v1.0.9 references: Zenodo, GitHub, PyPI (sandbox), IPFS, Software Heritage, GitHub Pages, Netlify, Hugging Face, OSF. Intended v2.0.0 is unreleased. Licensed Apache-2.0.

Source: projects/templates/template_template/manuscript/config.yaml

This deck is an instance of the same claim

- Every number on the previous two slides was read live from template_template's own manuscript/config.yaml and this monorepo's generated coverage report — not typed into this YAML file by hand.
- src/deck_tokens.py::build_deck_tokens is the function that did the reading; you can run it yourself and get the same numbers.
- A token-resolution audit fails this deck's own build if any of these values ever go missing.

Source: projects/templates/template_pitch_deck/src/template_pitch_deck/deck_tokens.py



The pattern generalizes



The full roster, not one cherry-picked example

• projects/templates/ holds all 25 public exemplars: template_active_inference, template_advanced_literature_review, template_autopoiesis, template_autoresearch_project, template_autoscientists, template_code_project, template_data_descriptor, template_eda_notebook, template_experiment_tree, template_formal, template_gold_refinement, template_literature_meta_analysis, template_madlib, template_methods_paper, template_newspaper, template_pitch_deck, template_pools_rules_tools, template_prose_project, template_redacted_report, template_registered_report, template_search_project, template_sia, template_storybook, template_template, template_textbook.

Source: docs/_generated/active_projects.md

One contract across all public exemplars

- template_active_inference is called out here only as one concrete instance to point to — every name in that list is an independently-built exemplar following the identical two-layer architecture, not a hand-picked best case.
- Neither exemplar's coverage gate or drift check required any special-casing to add — the contract is uniform across all 25 projects, and the folder itself is the evidence, not a claim about it.

Source: docs/_generated/active_projects.md

What makes an exemplar 'public'

- A single hand-edited registry (infrastructure/project/public_scope.py) lists every project name in public CI/documentation scope.
- Everything else under projects/ (working, ongoing, archive folders) stays local-only by construction, never accidentally published.
- An automated confidentiality audit fails CI if a private project path is ever tracked by mistake.



Governance and confidentiality

Public vs. private, by construction

- Only projects/templates/ is public and CI-gated; private working projects render locally through the same pipeline without ever being tracked.
- A dedicated audit (scripts/audit/check_tracked_all.py) enforces this on every push, not only at release time.
- The same separation pattern that lets a lab keep unpublished work private, while still exercising the identical tested pipeline, day to day.



Publishing surface

- A registry of 20 publishing platforms — Zenodo, arXiv, GitHub, PyPI, OSF, Software Heritage, Hugging Face, IPFS, and more — with per-platform automation where an API exists.
- Platforms without a submission API (arXiv) are documented as a manual step, not silently skipped or falsely marked complete.
- Every 'published' status in a project's README is compiled from that project's own config.yaml, not hand-written prose that can drift out of sync.



Reproducibility guarantees

What reproducible means here

- Fixed RNG seeds and MPLBACKEND=Agg for headless, deterministic figure generation.
- Two consecutive pipeline runs against the same repository state produce byte-identical rendered output.
- Coverage and test counts are read from a generated report (docs/_generated/COUNTS.md), never hardcoded into prose that could go stale.



How you would verify this deck's own claims

- Clone the monorepo, run the same pipeline command shown in this project's README, and regenerate this exact deck.
- Diff the regenerated PDF/PPTX against the committed output — a token-honesty test already does this as part of the project's own CI gate.
- Nothing in this pitch requires trusting the presenter; it requires running the pipeline.

Scientific integrity, by construction

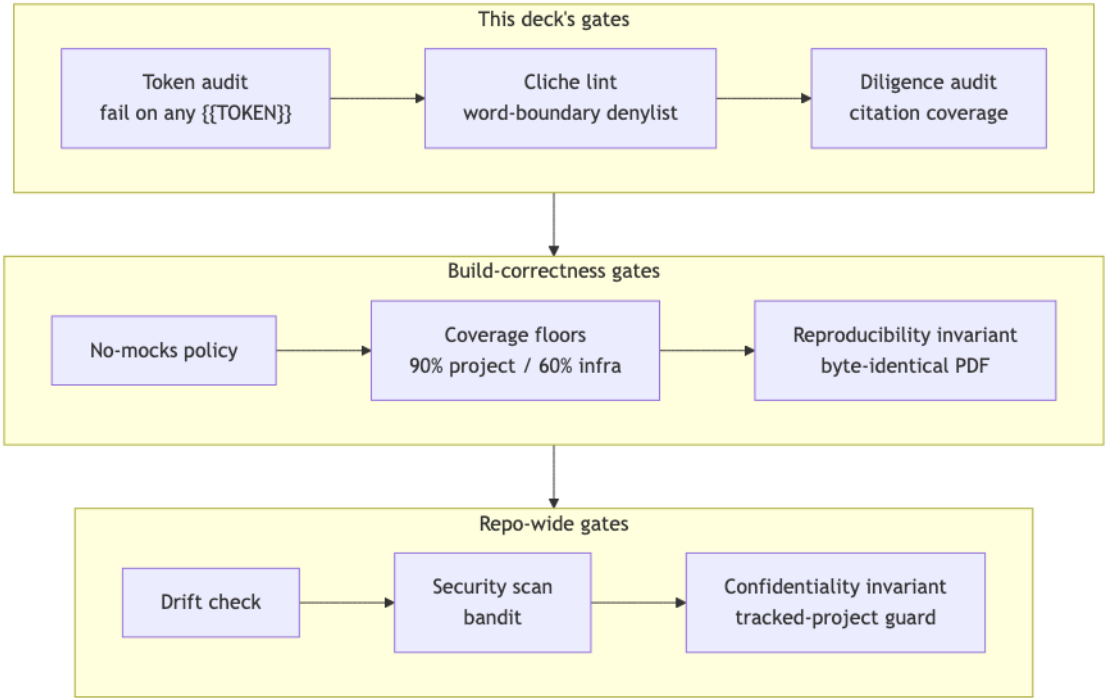
Independent gates, not one

- No-mocks testing policy, CI-enforced: scripts/audit/verify_no_mocks.py fails the build if any test imports MagicMock, mocker.patch, or unittest.mock — a checked gate, not a style preference.
- Coverage floors are enforced per project (90%) and for shared infrastructure (60%), not averaged away across the monorepo.
- A byte-level reproducibility invariant: two renders of identical content in the same environment produce an identical PDF — verified by a real, passing test, not assumed across every OS/toolchain combination.

Source: tests/infra_tests/rendering/test_slide_deck.py



The integrity stack, gate by gate



Source: projects/templates/template_pitch_deck/src/template_pitch_deck/token_resolution.py

Why gates instead of a style guide

- A style guide is a request; a gate is a build failure — the difference between 'we ask reviewers to check citations' and 'the render script exits non-zero if one is missing'.
- Each gate here targets one specific failure mode it was built to catch: a stale token, a cliche phrase, an uncited fact, a non-reproducible artifact.
- None of these gates existed before this exemplar needed them — each was added because a real defect surfaced during this project's own development, not speculatively.

Source: projects/templates/template_pitch_deck/src/template_pitch_deck/diligence_audit.py



What happens when a gate fails

- scripts/10_audit_deck_content.py exits non-zero on the first unresolved token or cliche hit — the render script never runs against unaudited content.
- scripts/30_audit_diligence.py exits non-zero if any fact-bearing slide lacks a source citation — a deck can be complete and still fail to ship.
- Fail-closed, not fail-open: an audit that cannot run is treated as a failure, never silently skipped and reported as a pass.

Source: projects/templates/template_pitch_deck/scripts/10_audit_deck_content.py

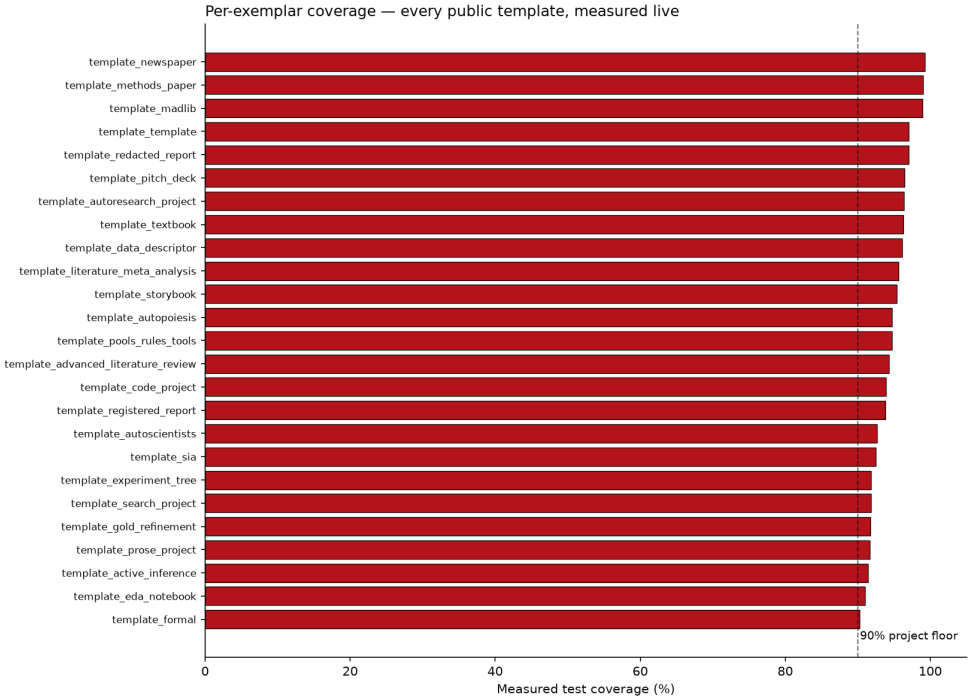


Coverage isn't a headline number

- Every one of 25 public exemplars is measured independently, not summarized into one repo-wide percentage.
- The chart on the next slide is generated from the same docs/_generated/COUNTS.md factsheet this deck's own 97.07% figure comes from — one source of truth, two presentations.

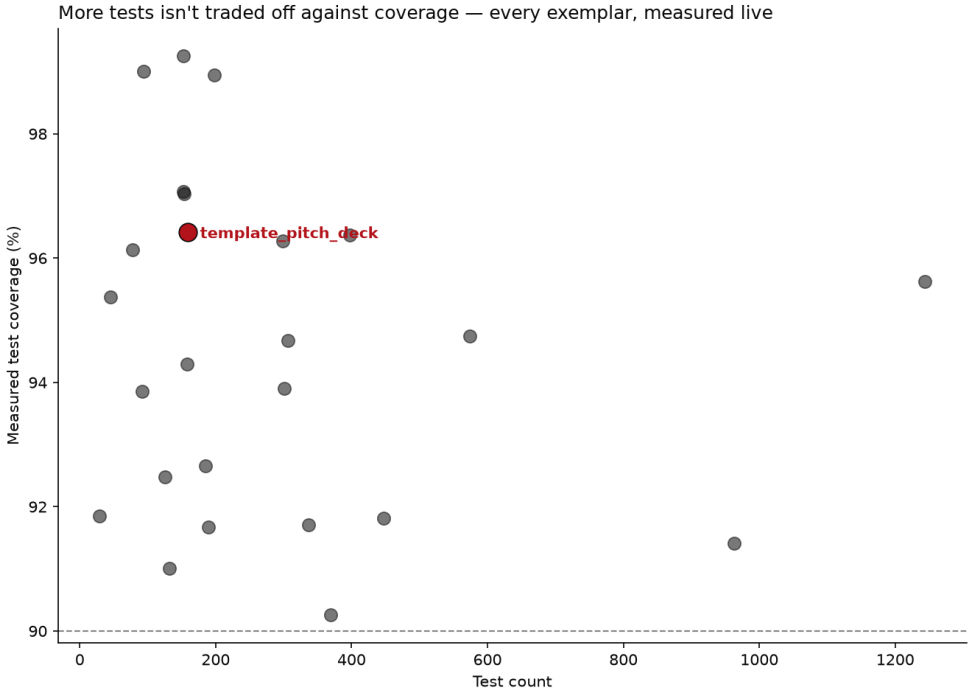
Source: docs/_generated/COUNTS.md

Per-exemplar coverage, measured live



Source: docs/_generated/COUNTS.md

Test volume and coverage aren't a tradeoff



Source: docs/_generated/COUNTS.md

This deck audits itself

- src/token_resolution.py fails the build on any unresolved double-curly-brace placeholder — the same discipline this slide's own template_template token was resolved by.
- src/cliche_lint.py and src/diligence_audit.py both run inside render_orchestration.py itself, before that deck length's own PDF/PPTX is written — a slide referencing a live-sourced token with no citation blocks that length's render, it does not just fail a separate, skippable check.
- None of these checks are generic infrastructure — they are this project's own src/, built specifically because a pitch deck is exactly the kind of document that tempts unverifiable claims.

Source: projects/templates/template_pitch_deck/src/template_pitch_deck/diligence_audit.py



The chain of custody: slide → QR → GitHub

- Every slide in this deck carries its own QR code, linking to a standalone, real Markdown page for that exact slide under output/slides_standalone/.
- Once this output/ directory is committed and pushed, a photo of a projected or printed slide can be traced back to its precise source page on GitHub — not just to the deck as a whole. Locally, the generated page already exists; the QR only resolves once it is actually published.
- Mechanically, this is the same source-citation system with one more hop: a citation links out to evidence; the QR links to the slide's own generated page, which repeats that same citation.

Source: projects/templates/template_pitch_deck/src/template_pitch_deck/standalone_slides.py

Cite this deck

- This pitch deck is its own exemplar in the same monorepo — it has its own src/, tests, and citable identity, distinct from the DOI of template_template, the project it pitches.
- This deck's own DOI: 10.5281/zenodo.21281509.
- That status is read live from this project's own manuscript/config.yaml at render time — the moment a real Zenodo deposit is recorded there, this slide's own text updates on the next regeneration, the same way every other fact in this deck does.
- This deck's DOI is deposited through the same infrastructure.publishing pipeline every other exemplar in this monorepo uses — nothing deck-specific to build, only a deposit to make.

Source: projects/templates/template_pitch_deck/manuscript/config.yaml

What's actually inside infrastructure/

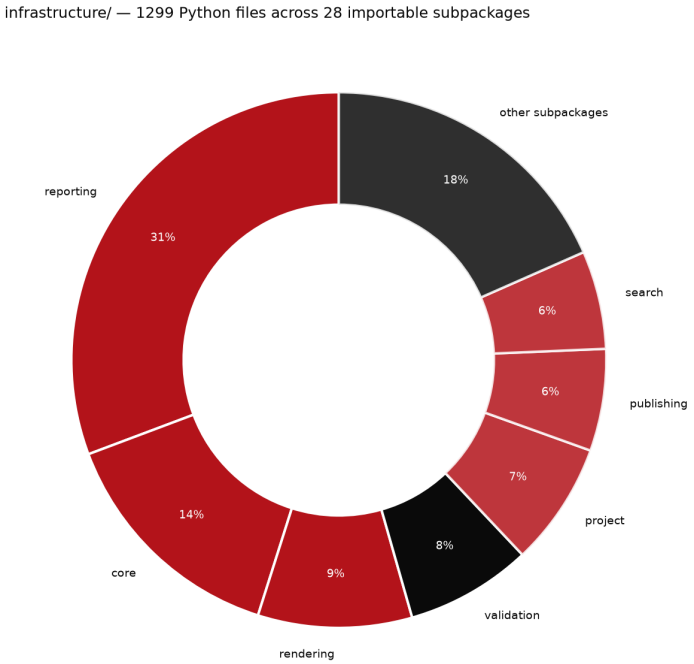


Not a black box

- 28 importable subpackages, 1302 tracked Python files total under infrastructure/ — a real inventory anyone can `git ls-files` themselves, not a claimed abstraction.
- The largest single subpackage, reporting, holds 399 files — still a minority of the whole, not a monolith wearing a modular label.
- Every number on this slide is computed live by src/infra_facts.py at render time, reusing the exact same counters docs/_generated/COUNTS.md is built from.
- The next chart narrows the total to importable subpackages, excluding 3 top-level file(s): infrastructure/__init__.py and mcp_server.py.

Source: projects/templates/template_pitch_deck/src/template_pitch_deck/infra_facts.py

infrastructure/, by subpackage



Source: projects/templates/template_pitch_deck/src/template_pitch_deck/infra_facts.py

Why partition it this way

- Each subpackage boundary matches a real reuse boundary: rendering/, validation/, publishing/, provenance/ each solve one generic problem every exemplar needs, independent of any project's domain logic.
- A change inside one subpackage cannot silently reach into another without an explicit import — the boundary is enforced by the drift check, not just convention.
- This is the same two-layer split described earlier in this deck, now shown at the resolution of individual subpackages instead of the whole layer.



Roadmap



What comes next for this exemplar

- Extend token-sourced facts beyond one pitch subject to any exemplar in the roster, so a new deck for a different project needs only new content YAML, not new code.
- Add a second validated pitch (a broader meta-science-group deck) reusing the same rendering and audit pipeline.
- Track deck-generation coverage and drift in the same CI gates that already watch every other public exemplar.



Where this could go next for your organization

- A grant-report deck generated the same way a manuscript is — bound to the same project data your team already tracks.
- A recurring quarterly-update deck that regenerates instead of being rebuilt from scratch each cycle.
- A shared content schema across your organization's decks, so a house style survives staff turnover.



The ask

- Adopt this pattern for your own methods papers: bind every reported number to the code that produced it, not to a hand-typed table.
- Pilot it on one existing manuscript in your group before committing to a full rewrite.
- Happy to pair with your team through a first regeneration cycle and help define your own introspection surface, if useful — no prior engagements to point to yet, this pattern is newly forkable.



Funding & consulting

- The code itself is available to adopt directly today — real, tested, and licensed for reuse, no engagement required to start.
- Open to funding conversations for a dedicated integration (a new pitch subject, a distinct theme, a deeper introspection surface) — no such engagement exists yet, this is an invitation to scope one, not an established program.
- Open to scoping a consulting engagement on request — pairing through a first regeneration cycle, or adapting the two-layer architecture to an existing codebase outside this monorepo. No such engagement has happened yet; this is an invitation, not a track record.



Appendix

Glossary

- Autopoietic — a system that regenerates its own description from its own live state, rather than from a fixed snapshot.
- Thin orchestrator — a script that coordinates infrastructure/ and src/ calls but contains no business logic itself.
- Coverage floor — the minimum percentage of source lines a test suite must exercise before the pipeline is allowed to proceed.
- No-mocks policy — a testing rule requiring real files, real data, and real computation instead of simulated objects.



Citation and contact

- Concept DOI: 10.5281/zenodo.20419007.
- Repository: docxology/template_template, licensed Apache-2.0.
- This deck itself: projects/templates/template_pitch_deck in the same monorepo.

Source: projects/templates/template_template/manuscript/config.yaml

Questions

Research infrastructure that documents itself — a case study for meta-science and science-integrity teams

